Daftime — Handoff Développeur

> ***Stack*** *: Next.js 14.2 (App Router) · React 18 · Tailwind CSS 3.4 · TypeScript · Framer Motion 12 · Cobe (globe WebGL).*
>
> ***Repo*** *:* `tinkode92/daftime-test` *·* ***Branche prod*** *:* `main` *·* ***Hébergement*** *: Vercel.*

Bienvenue. Ce document explique l'architecture du site Daftime, en particulier la **gestion du multi-pays / multi-langues** et comment faire évoluer les 3 sites (AE, FR, PT) indépendamment.

---

1. Architecture générale

```
daftime-test/
├── app/
│   ├── layout.tsx           # Root layout: monte ScrollProgress + CountryGate
│   ├── globals.css          # Tailwind base + keyframes (marquee, fade-up, halo)
│   ├── page.tsx             # 🇦🇪  AE — homepage
│   ├── fr/page.tsx          # 🇫🇷  FR — homepage France
│   ├── pt/page.tsx          # 🇵🇹  PT — homepage Portugal
│   └── resources/
│       ├── podcast/page.tsx
│       └── guide-2026/page.tsx
├── components/
│   ├── Hero.tsx · Services.tsx · ServiceTabs.tsx · Blog.tsx ·
│   ├── Testimonials.tsx · GuideCTA.tsx · BookConsultationCTA.tsx ·
│   ├── Footer.tsx · Navigation.tsx · BrandMarquee.tsx ·
│   ├── CountryGate.tsx          # popup pays + langue
│   ├── GuideDownloadModal.tsx   # popup PDF guide
│   ├── LearnMoreModal.tsx       # popup contact (Hero CTA)
│   ├── Globe.tsx                # globe WebGL (cobe)
│   ├── Reveal.tsx               # IntersectionObserver-based reveal
│   └── motion/                  # primitives Framer Motion
│       ├── WordReveal.tsx
│       ├── AnimatedCounter.tsx
│       ├── MagneticButton.tsx
│       ├── TiltCard.tsx
│       └── ScrollProgress.tsx
├── lib/
│   ├── translations.ts          # dictionnaires EN / FR / PT
│   └── useEffectiveLocale.ts    # hook pour résoudre la langue active
├── public/assets/               # SVG, images, illustrations
├── tailwind.config.ts           # tokens (couleurs daftime-*, h-display, label-mono…)
└── next.config.mjs
```

---

2. Le modèle COUNTRY × LANGUAGE — le cœur du système

Daftime a **2 axes indépendants** de personnalisation :

Axe	Rôle	Pilote	Stockage
Country	Quel bureau Daftime / quelle juridiction	URL `/`, `/fr`, `/pt`	`localStorage["daftime-country"]`
Language	Langue d'affichage du texte	Aucun (préférence pure)	`localStorage["daftime-language"]`

Les deux sont **découplés** : un visiteur basé à Dubaï peut lire le site en français, un Portugais peut le lire en anglais, etc.

COMBINAISONS POSSIBLES

Country	Languages disponibles
🇦🇪 AE	EN, FR
🇫🇷 FR	EN, FR
🇵🇹 PT	EN, PT

> ⚠️ ***PT n'est pas dispo*** *sur AE et FR (le portugais ne sert que pour le bureau de Lisbonne). Cette règle est appliquée dans* `components/CountryGate.tsx` *via* `isLanguageAvailable(country, lang)`.

VISIBLE SIDE-EFFECTS

- Sur **country = AE uniquement**, les prix AED des plans d'Accounting sont affichés (Basic AED 999.99/mois, etc.). Hors AE, la ligne de prix est masquée. → Logique dans `components/ServiceTabs.tsx` : `useStoredCountry()` et le prop `showPrice` du `PricingCard`.
- Le **logo en haut à gauche** et toutes les ancres du nav (`/#what`, `/#contact`, etc.) pointent vers la home du **pays courant** (`/`, `/fr` ou `/pt`).

---

3. Les 3 pages homepage — architecture indépendante

C'est le point le plus important pour faire évoluer le projet.

AUJOURD'HUI

Les 3 fichiers `app/page.tsx`, `app/fr/page.tsx`, `app/pt/page.tsx` sont **strictement indépendants** : 3 routes Next.js distinctes, chacune avec ses propres metadata (`<title>`, `<description>`).

Aujourd'hui ils importent le même set de 8 composants dans le même ordre, juste avec un `locale` prop différent :

```
// app/fr/page.tsx
import Hero from "@/components/Hero";
import Services from "@/components/Services";
// …

export default function HomeFR() {
  return (
    <main className="min-h-screen w-full bg-white">
      <Hero locale="fr" />
      <Services locale="fr" />
      <ServiceTabs locale="fr" />
      <Blog locale="fr" />
      <Testimonials locale="fr" />
      <GuideCTA locale="fr" />
      <BookConsultationCTA locale="fr" />
      <Footer locale="fr" />
    </main>
  );
}
```

POUR FAIRE DIVERGER UN SITE DEMAIN

C'est là que le découplage paie : tu **édites un seul des 3 fichiers**, sans rien casser sur les autres.

```
// Exemple : faire que /fr ait une section différente
// app/fr/page.tsx

export default function HomeFR() {
  return (
    <main className="min-h-screen w-full bg-white">
      <Hero locale="fr" />
      <ServicesFR />               {/* nouveau composant FR-only */}
      {/* On retire ServiceTabs */}
      <BlogFR />                   {/* version France-only avec articles dédiés */}
      <Testimonials locale="fr" /> {/* on garde le composant partagé */}
      <BookConsultationCTA locale="fr" />
      <Footer locale="fr" />
    </main>
  );
}
```

Aucune modification de `/app/page.tsx` n'affectera /fr ou /pt, et inversement.

---

4. Le système de traduction

LE DICTIONNAIRE CENTRAL

Tout le texte du site vit dans un seul fichier : `lib/translations.ts`. Structure :

```
const en = { nav: {…}, hero: {…}, services: {…}, /* … */ };
const fr: typeof en = { /* … */ };
const pt: typeof en = { /* … */ };

export const translations = { en, fr, pt };
export function t(locale: Locale): Translation { /* … */ }
```

Pour ajouter ou modifier un texte, tu modifies les 3 objets `en`, `fr`, `pt`. TypeScript s'assure que tu n'oublies pas une clé (le typage `typeof en` force la même forme pour fr et pt).

LE HOOK `USEEFFECTIVELOCALE`

C'est lui qui résout la langue **réelle** affichée par un composant. Il combine 2 sources :

1. La prop `locale` passée par la page (= le défaut SSR, suit l'URL)
2. `localStorage["daftime-language"]` (= la préférence utilisateur)

```
// lib/useEffectiveLocale.ts
export function useEffectiveLocale(serverLocale: Locale): Locale {
  const [locale, setLocale] = useState<Locale>(serverLocale);
  useEffect(() => {
    const sync = () => {
      const stored = readStored();
      setLocale(stored ?? serverLocale);
    };
    sync();
    window.addEventListener("daftime-locale-changed", sync);
    return () => window.removeEventListener("daftime-locale-changed", sync);
  }, [serverLocale]);
  return locale;
}
```

Dans chaque composant :

```
export default function Hero({ locale = "en" }: { locale?: Locale }) {
  const effectiveLocale = useEffectiveLocale(locale);
  const tr = t(effectiveLocale).hero;
  return <h1>{tr.title}</h1>;
}
```

> 💡 ***Pourquoi ce double système ?*** *SSR (et le premier paint) utilisent la prop, donc l'HTML servi est cohérent avec l'URL. Une fois le client hydraté, si l'utilisateur a une préférence langue stockée, le composant re-render dans cette langue. Cette approche évite les hydration mismatches.*

QUAND LA POPUP COUNTRYGATE CONFIRME UN CHOIX

`components/CountryGate.tsx` → `confirm()` :

1. Sauve `daftime-country` et `daftime-language` dans localStorage
2. Dispatch `window.dispatchEvent(new CustomEvent("daftime-locale-changed"))`
3. Tous les `useEffectiveLocale` écoutent cet event et re-render

Résultat : changer la langue dans la popup re-render immédiatement tout le site sans reload.

---

5. La popup CountryGate

Fichier : `components/CountryGate.tsx`. Montée globalement dans `app/layout.tsx`.

COMPORTEMENT

- **Première visite** : aucune valeur en localStorage → la popup s'ouvre. L'utilisateur doit sélectionner pays + langue puis cliquer **Continue**.
- **Visites suivantes** : popup invisible. La page lit les choix stockés et applique :
  - URL ↔ pays : si stored country ≠ URL country, redirige vers la home du pays stocké (ex: arrivée sur `/` avec stored=FR → push `/fr`).
  - Texte ↔ langue : applique la langue stockée via `useEffectiveLocale`.
- **Ré-ouverture manuelle** : depuis n'importe quel composant, `import { openCountryGate } from "@/components/CountryGate"` puis `openCountryGate()`. Utilisé par la pill 🇬🇧 EN du navbar.

RÈGLES INTERNES

- La langue **PT n'est dispo que si le pays = PT** (filtre dans `isLanguageAvailable`).
- Quand on change le pays, si la langue active devient invalide, elle retombe automatiquement sur **EN**.

---

6. Animations (Framer Motion + CSS)

Toutes les animations vivent dans `components/motion/` ou inline dans les composants. Vue d'ensemble :

Animation	Composant	Effet
Reveal au scroll	`Reveal.tsx` (IntersectionObserver)	Fade-up, slide, scale au passage en viewport
Mot par mot	`motion/WordReveal.tsx`	Reveal de titres avec blur → focus
Compteur	`motion/AnimatedCounter.tsx`	Tweens un nombre 0 → 12000 quand visible
Bouton magnétique	`motion/MagneticButton.tsx`	Suit le curseur en spring
Tilt 3D	`motion/TiltCard.tsx`	Inclinaison hover, glare optionnel
Barre de progression	`motion/ScrollProgress.tsx`	Top bar jaune qui se remplit au scroll
Marquee CSS	`globals.css` `.marquee-track`	Animation `@keyframes marqueeX`, pause au hover
Float / spin / halo	`globals.css`	Classes utilitaires `.float-slow`, `.spin-slow`, `.halo-pulse`

Les sections `/resources/guide-2026/GuideSections` et `GuideAudience` ont des animations **scroll-driven** (timeline jaune qui se remplit, bullets qui s'allument en cascade).

> 🌱 ***Accessibilité*** *:* `globals.css` *respecte* `@media (prefers-reduced-motion: reduce)` *pour désactiver les animations.*

---

7. Workflow de déploiement (Vercel)

- Push sur `main` → déploiement automatique en théorie.
- ⚠️ **Le webhook GitHub→Vercel est intermittent**. Si un push n'a pas lancé de build, utiliser le **Deploy Hook** (Vercel → Settings → Git → Deploy Hooks). Coller l'URL dans le

navigateur ou `curl -X POST <url>` force un build sur le HEAD actuel de main.
- Cliquer **Redeploy** sur un ancien déploiement rebuilde **ce commit-là**, pas le HEAD. Toujours préférer le Deploy Hook pour rebuild la dernière version.

CACHE CDN

Les assets dans `public/assets/` sont cachés agressivement par le CDN Vercel. Si tu modifies un SVG, **renomme le fichier** (ex: `folder.svg` → `folder-v2.svg`) et mets à jour les imports — sinon les vieux SVG restent servis.

---

8. Points d'attention pour le repreneur

BUILD & LINT

```
npm install
npm run dev      # dev server sur :3000
npm run build    # build production (à passer avant chaque commit majeur)
npm run lint     # ESLint
```

Le build doit toujours passer (TypeScript strict + ESLint). Un `Failed to compile` casse le déploiement Vercel.

MODIFIER UN TEXTE

1. Ouvrir `lib/translations.ts`
2. Localiser la section concernée (par ex. `hero`, `services`, `blog`)
3. Modifier la chaîne dans **les 3 objets** `en`, `fr`, `pt` (TypeScript ne te laissera pas oublier)
4. Pas besoin de toucher aux composants, ils relisent automatiquement

MODIFIER LE GLOBE HERO

`components/Globe.tsx` (basé sur `cobe`). Les markers (Dubai, Paris, Lisboa) sont en haut du fichier. La taille / glow / dpr sont aussi ajustables. La rotation auto + drag est gérée par un `requestAnimationFrame` interne.

AJOUTER UN SOUS-DOMAINE OU UNE NOUVELLE LANGUE

- Ajouter une clé dans `Locale` (`lib/translations.ts`)
- Ajouter la traduction complète dans le dictionnaire

- Optionnel : créer `app/<lang>/page.tsx` pour la nouvelle URL
- Mettre à jour la popup CountryGate (`countries` , `languages` , `isLanguageAvailable`)

PAGES RESOURCES (`/RESOURCES/PODCAST` , `/RESOURCES/GUIDE-2026`)

Pour l'instant, ces pages sont **uniquement en anglais** et ne suivent pas le système de traduction (pas de prop `locale`). Si tu veux les internationaliser plus tard :

1. Ajouter les sections dans `translations.ts`
2. Passer locale aux composants `Guide*` et `Podcast*`
3. Optionnel : créer `/fr/resources/...` et `/pt/resources/...`

---

9. Roadmap suggérée

Priorité	Tâche
🔴 Haute	Déposer le vrai PDF du guide à `public/assets/daftime-guide-2026.pdf` (le bouton "Get access" du modal pointe dessus)
🟡 Moyenne	Ajouter URLs LinkedIn dans `Footer.tsx` (lignes Instagram et Youtube déjà câblées)
🟡 Moyenne	Câbler un endpoint email pour les formulaires `LearnMoreModal` et `GuideDownloadModal` (aujourd'hui juste front)
🟢 Basse	Faire diverger le contenu de /fr et /pt selon les besoins business (édition des `app/fr/page.tsx` / `app/pt/page.tsx`)
🟢 Basse	Internationaliser les pages `/resources/...`
🟢 Basse	Traduire `LearnMoreModal.tsx` (encore en EN seulement)

---

10. Crédits

Setup initial du design Figma → implémentation par Claude Code. Toute l'architecture i18n + animations + popups est documentée par les commits sur la branche `main` (history git détaillé).

> ***Bonne reprise — toutes les conventions sont là pour t'aider à itérer vite sans casser l'existant.***